

VoteSentinel: Post-quantum Verifiable Coercion-resistant Voting with Dispute Resolution

Navid Abapour and Constantin Cătălin Drăgan

Surrey Centre for Cyber Security, University of Surrey, UK
{n.abapour, c.dragan}@surrey.ac.uk

Abstract. Electronic voting systems have to satisfy challenging requirements: balancing the need for transparency while ensuring vote secrecy. These systems need to resist both classical and quantum adversaries. Specifically, no work to date attests to a scheme with provably strong privacy guarantees and post-quantum security simultaneously. Classical systems, such as Helios and Microsoft ElectionGuard, become vulnerable to clash attacks, ballot stuffing, and coercion through confirmation codes. Generally, electronic voting is widely used, and designing post-quantum systems is needed, but post-quantum voting schemes not only suffer from a lack of formal ballot/participation privacy proofs from strong definitions, but also have no guarantee for coercion resistance.

In this work, we introduce **VoteSentinel**, the first voting scheme that achieves a wide range of security properties without using expensive zero-knowledge proofs or signature schemes under a single framework. Our security analysis shows that **VoteSentinel** closes a large array of vulnerabilities previously discovered in other schemes, while supporting ranked-choice and re-voting. Moreover, **VoteSentinel** brings coercion resistance and accountability properties at the same time.

Keywords: Applied Cryptography, Electronic Voting, Ballot Privacy, Coercion Resistance, Post-Quantum Cryptography

1 Introduction

The idea of electronic voting is to modernize democratic processes through more accessibility and efficiency; however, designing schemes that can meet a wide range of security and privacy requirements remains one of the challenging problems in cryptography [1]. The paradox lies between transparency toward public verifiability and a high level of privacy for individual votes [2]. This challenge gets more complicated by the advent of quantum computers, which will be in a position to retroactively break today’s classically encrypted ballots [3]. In spite of the fact that classical schemes have made significant progress in verifiability, they remain vulnerable to sophisticated attacks. Some Helios variants suffer from clash attacks, where adversaries can force ballot collisions [4]. ElectionGuard’s confirmation codes [5] enable coercion through forced disclosure.

Moving towards post-quantum cryptography introduces additional complexity through larger key sizes and more intricate algebraic structures [6]. Transition to post-quantum e-voting needs to happen as soon as possible, because of the Store

Now, Decrypt Later attack [7], in which encrypted data is harvested today with the intention of decrypting it in the future, posing a big risk to information that must remain secure long term. Existing lattice-based voting schemes, such as NTRU Mix-Net [8], provide quantum resistance but lack formal ballot/participation privacy proofs from strong definitions and fail to address other essential properties, e.g., dispute resolution, accountability, etc.

Most voting schemes are designed such that no entity can learn the content of ballots (*ballot privacy* [9]), participation patterns remain hidden from observers (*participation privacy* [10]), voters can verify their ballots were recorded and anyone can verify correct tallying (*verifiability* [11]), the coercer cannot know whether voter followed his instruction or not (*coercion resistance* [12]), ballot secrecy gets preserved against computationally unbounded future adversaries (*everlasting privacy* [13]), only the most recent vote from each voter influences the final tally (*strong non-reusability* [14]), voters can update their choices multiple times (*re-voting* [15]), conflicts between voters and authorities are resolvable through cryptographic evidence (*dispute resolution* [16]), misbehaving parties can be identified and held responsible (*accountability* [17, 18]), the protocol execution is indistinguishable from interaction with a trusted party (*ideal functionality* [19]), voters cannot construct proofs of their choices to third parties (*receipt-freeness* [20]), the election can complete even when some trustees lose their secret keys (*key-loss resilience* [21]), and voters can express preferences by ranking candidates in some cases (*ranked-choice voting* [22]). However, currently, there is no voting scheme that can simultaneously provide all of these properties; there are many schemes that each target one or a very limited number of these properties, not all of them in one place, as it is difficult to provide more than 2 of these properties at the same time [6], and it is challenging.

1.1 Contributions

These gaps motivated us to design a voting system that integrates a wide range of security properties, as Table 1 shows, instead of designing a core scheme and then attempting to achieve different separated properties by repeatedly designing some variants of it, as most classical approaches do. First, we recall and identify some attack vectors and conceptual weaknesses of the existing classical and post-quantum voting schemes, such as clash attacks, as well as a lack of strong formal privacy proofs. Next, we present **VoteSentinel**, a verifiable voting scheme that not only achieves all of the aforementioned properties together at the same time through a unified threshold-homomorphic framework with nullifier-based ballot resolution, but also brings post-quantum-level security, combining ML-KEM for secure channels, BFV for homomorphic tallying, and VSS for distributed trust, our scheme addresses vulnerabilities in existing schemes. Then, based upon this, we prove the privacy of **VoteSentinel** for every claimed property under the standard lattice assumptions.

1.2 Related Works

We go through some of the existing electronic voting schemes, analyzing classical systems (Helios [38], Belenios [39], ElectionGuard [5]) and post-quantum

Property/PQE-voting Scheme	Epoque [23]	NTRU Mix-Net [8]	FOO-based [3]	Code-based [6]	VoteSentinel (Ours)
Computational Assumption	RLWE, RSIS	RLWE, MSIS, RSIS	RLWE, OM-RSIS, MSIS	Dual-LPN	RLWE, MLWE
Core Building Blocks	IBE [24] + Com [25] + ZKP [26]	NTRU [27] + Shuffle [28] + ZKP [29]	BS [30] + PKE [31]	Com [6, 32] + ZKP [6, 32] + Shuffle [6]	ML-KEM [33] + BFV [34] + VSS [35]
Ballot Privacy Proof [19]	✓	×	×	×	✓
Participation Privacy Proof [10]	×	×	×	×	✓
Individual Verifiability [11]	✓	✓	Partial	✓	✓
Universal Verifiability [11]	✓	✓	✓	✓	✓
Everlasting Privacy Proof [36]	×	×	×	×	✓
Receipt-Freeness [20]	Partial	✓	×	Partial	✓
Accountability [18]	✓	×	×	×	✓
Ideal Functionality Proof [19]	×	×	×	×	✓
Coercion Resistance [20]	×	×	×	×	✓
Dispute Resolution [16]	×	×	×	×	✓
Vote Updating/Revoting [15]	×	×	×	×	✓
Homomorphic Tallying [37]	✓	×	×	×	✓
Trusted Setup	Required	Partial Required	Required	Not Needed	Required
Honest Majority Talliers	Required	Required	Required	Required	Required
Trusted Bulletin Board	Required	Required	Required	Required	Required

Table 1: Comparison of post-quantum e-voting schemes [3, 6, 8, 23] by structural components, privacy properties, trust assumptions, and features. ✓ indicates satisfying an specific definition, and × shows not providing a desirable feature.

approaches (Epoque [23], NTRU Mix-Net [8], FOO-based [3], Code-based [6]) to identify fundamental vulnerabilities and limitations that VS addresses through its lattice-based dual-chain architecture.

Classical Voting Approaches Starting with Helios, as a well-studied scheme, it is a web-based voting system that prioritizes verifiability over coercion resistance, making it suitable for low-coercive environments [38]. However, several variants of Helios are vulnerable to Clash Attacks [4], and upon compromise of the server, they can be attacked by Ballot Stuffing, and Individual Verifiability violations [40] and other privacy attacks [41] as well. Another voting system known as Belenios is a web-based verifiable voting system led its development by Stéphane Glondou that improves Helios through formal verification [39]. Clash Attacks, Ballot Re-ordering Attacks, and Strong Ballot Stuffing [40] under the corrupted registrar are some of the Belenios’ vulnerabilities. Developers have published further issues with Belenios regarding fragile vote privacy and the mixnet tally [42], as well.

As an open-source framework developed by Microsoft Research, ElectionGuard also enables verifiable electronic voting systems. It allows third parties to independently verify the integrity of election results without compromising voter privacy [5]. ElectionGuard gives the ability for voters, candidates, observers, and independent organizations to verify election results. Despite all the new components, ElectionGuard can be considered, in essence, a modification of Cramer et al. [43]. Although ElectionGuard is a software development kit, it is not compatible with all existing modes of casting votes.

Helios has been well-studied during the past decades, and Belenios is ultimately a variant of it; the weaknesses of these two schemes have already been observed. In the following, we present our findings on the downsides of ElectionGuard.

ElectionGuard does not support ranked-choice voting. Additionally, in ElectionGuard, voters receive confirmation codes $c_i = H(E_{pk}(v_i))$ for their encrypted ballots, which enables coercion: adversaries with decryption capabilities can demand these codes to verify compliance. Public disclosure of confirmation codes compromises ballot privacy, as c_i uniquely links voters to their encrypted ballots $E_{pk}(v_i)$, enabling unauthorized decryption and vote recovery [44]. In voting systems that use this framework with cumulative voting, the discrete logarithm sizes grow beyond small values when voters cast multiple votes for a candidate,

and ElectionGuard’s precomputed DLog tables are susceptible to attacks, expose keys, lack generality, and suffer from efficiency trade-offs [44]. The secrecy of ballots in ElectionGuard is tied to a single value known as the ballot nonce [5]. These values must be carefully protected because exposure of this random value can directly lead to the adversary decrypting votes¹. Up to today, ElectionGuard has not been formalized yet; also, no proof for ballot privacy, or receipt-freeness, or participation privacy has been provided for it.

None of the Helios, Belenios, and ElectionGuard provides solutions for dispute resolution. ElectionGuard addresses minor complaints during the election initialization process, but does not provide formal and robust answers. Furthermore, to our knowledge, none of the classical and post-quantum schemes give any robust result recoverability for scenarios like IACR 2025 incident [21, 46, 47], and existing schemes have weak resilience to key-loss.

The next section provides an overview of the state-of-the-art in e-voting schemes that are resistant to attacks by quantum computers, similar to **VoteSentinel**.

Post-quantum Voting Schemes The voting systems that are resistant to quantum attacks have diverse structures. Starting with NTRU Mix-Net [8], it employs NTRUEncrypt with single-element ciphertexts and verifiable mixing. The NTRU parameters must be carefully chosen to avoid the overstretched regime where $q > 0.0058 \cdot \sigma^2 \cdot d^{2.484}$ (with d the polynomial degree, q the modulus, and σ the noise distribution width) enables subfield lattice attacks, and the scheme’s privacy fundamentally relies on the assumption that at least one mix server remains honest without providing detection mechanisms for malicious mixing beyond proof verification.

FOO-based Voting [3] is another scheme that combines threshold lattice-based blind signatures with Dual-Regev encryption for everlasting privacy via perfectly hiding commitments. Authorities generate threshold shares for signature and encryption keys using Shamir secret sharing over polynomial rings. The one-more unforgeability of the underlying lattice-based blind signature scheme remains conjectural, as noted by Hauck et al. [48], potentially allowing adversaries to obtain additional valid signatures beyond those authorized; furthermore, the scheme requires a perfectly anonymous channel for ballot submission, which, if compromised, breaks the everlasting privacy guarantee despite the perfectly hiding commitments.

The Code-based scheme [6] employs the Learning Parity with Noise (LPN) assumption for commitments and ZK proofs, constituting the first code-based verifiable e-voting system. The scheme relies on the relatively less-studied dual-LPN assumption for the commitment binding property, which, unlike standard LPN, lacks an extensive cryptanalytic background; additionally, the concrete parameter selection for achieving 128-bit security against Information Set Decoding attacks remains uncertain, given recent advances in code-based cryptanalysis [49] and the lack of standardized parameter recommendations.

Epoque employs the Agrawal-Boneh-Boyen Identity-Based Encryption for verifiability without mix-nets. Unlike other similar schemes, Epoque shows accountability and privacy proof, but even assuming their proof is valid, developers use

¹ EG v2.1 [45] somehow addresses this issue, but at the cost of encrypting the nonce itself.

a very limited definition of ballot privacy known as "*Vote Privacy*" [50]. It has been shown that this limited definition restricts the class of protocols or privacy breaches [19]. Specifically, this definition may lead to inconsistent results when applied to protocols that use natural result functions, such as the majority function [37]. Additionally, the definition may not apply to protocols that output auxiliary data beyond just the result, such as proofs of correct tallying that most protocols produce, e.g., ElectionGuard [5]. The definition also appears to only detect specific types of privacy breaches, such as determining whether two voters cast the same vote, while missing other potential privacy violations [19]. As the authors of Epoque acknowledge, their scheme does not provide coercion-resistance².

Apart from the aforementioned disadvantages, none of these four post-quantum schemes [3, 6, 8, 23] provide wide-range proofs for their the cryptographic privacy properties that an e-voting system should have³, as mentioned in Section 1.

As Figure 1 shows, in Section 3 we present a voting scheme **VoteSentinel** that satisfies a broad range of security properties simultaneously with many additional capabilities (re-voting, ranked voting, etc.) designed using only three primitives as core (ML-KEM)+BFV+VSS.

A number of other post-quantum schemes have been proposed [52–57], but their underlying structures differ from the schemes discussed above and are therefore not directly comparable to ours, or they are designed for more constrained, more strict, or application-specific scenarios. For example, although the scheme proposed by Abapour et al. [52] includes a proof of ballot privacy, it is exclusively for economic structures and, due to the utilization of blockchain, does not have enough flexibility to be used for other purposes [58] like Helios.

2 Preliminaries

We define several mini-voting schemes separately to demonstrate the property definitions independently, thereby avoiding overlap. Let λ denote the security parameter. $[n]$ denotes $\{1, \dots, n\}$ as the indices of all n trustees who receive shares. We write $x \leftarrow \$S$ for uniform sampling from set S , and $x \leftarrow \mathcal{D}$ for sampling from probability distribution $\mathcal{D}$. We denote by $\text{negl}(\lambda)$ a negligible function in λ . Let $R_q = \mathbb{Z}_q[X]/(X^n + 1)$ be the quotient ring of polynomials where n is a power of 2.

Definition 1 (Hash Function [59]). *A collision-resistant deterministic map $H : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$, from arbitrary-length inputs to fixed-length λ -bit outputs, where finding distinct x, x' such that $H(x) = H(x')$ is computationally infeasible.*

Definition 2 (Commitment Scheme Com [60]). *A commitment scheme $\text{Com} = (\text{Cmt}, \text{Opn})$ consists of two sub-procedures for obligation of parties:*

- $\gamma \leftarrow \text{Cmt}(m; r)$: On input message m and randomness $r \in \{0, 1\}^\lambda$, outputs commitment $\gamma = H(m \| r)$.
- $\{0, 1\} \leftarrow \text{Opn}(\gamma, m, r)$: Outputs 1 iff $\gamma = H(m \| r)$.

² Even the definition that has been used to show Epoque is verifiable [51] is not strong as shown in [11] and does not distinguish between attack scenarios where honest voters' ballots are dropped (deleted) versus those where ballots are actually modified to contain different choices, even though ballot modification has a significantly stronger negative impact on election outcomes than ballot deletion.

³ Code-based Voting [6] shares some claims but no proof of ballot/participation privacy.

The scheme is binding (finding m, r, m', r' with $(m, r) \neq (m', r')$ but $H(m||r) = H(m'||r')$ is infeasible under collision resistance of H) and hiding (given γ , distinguishing which message was committed is infeasible when modeled in ROM).

Definition 3 (Key Encapsulation [33]). The key encapsulation mechanism ML-KEM = (KG, Enc, Dec) based on Module Learning With Errors MLWE [61] consists of:

- $(ek, dk) \leftarrow \text{KG}(1^\lambda)$: Outputs encapsulation key ek and decapsulation key dk .
- $(K, \psi) \leftarrow \text{Enc}(ek)$: Outputs shared key $K \in \{0,1\}^\lambda$ and ciphertext ψ .
- $K \leftarrow \text{Dec}(dk, \psi)$: Outputs shared key K or $\perp$.

Definition 4 (Homomorphic Encryption BFV [34]). A homomorphic encryption scheme BFV = (KG, Enc, Dec, Add) based on Ring Learning With Errors RLWE [61] with plaintext space R_p and ciphertext space R_q^2 consists of:

- $(pk, sk) \leftarrow \text{KG}(1^\lambda)$: Outputs public key $pk = (a, b) \in R_q^2$ and secret key $sk \in R_q$.
- $c \leftarrow \text{Enc}(pk, m; \rho)$: On input $m \in R_p$ and randomness ρ , outputs ciphertext $c = (c_0, c_1) \in R_q^2$.
- $m \leftarrow \text{Dec}(sk, c)$: Outputs plaintext $m \in R_p$.
- $c' \leftarrow \text{Add}(c_1, c_2)$: Outputs c' satisfying $\text{Dec}(sk, c') = \text{Dec}(sk, c_1) + \text{Dec}(sk, c_2)$.

Definition 5 (Verifiable Secret Sharing VSS [35]). A (t, n) -threshold verifiable secret sharing scheme VSS = (Shr, Vf, Rec, PDec, Comb) over R_q based on post-quantum hash-based commitment [62] consists of:

- $(\{u_i\}_{i \in [n]}, \{W_j\}_{j \in [0, t-1]}) \leftarrow \text{Shr}(s, t, n)$: Samples $\{a_j\}_{j=1}^{t-1} \leftarrow \$_{R_q^{t-1}}$, defines polynomial $f(X) = s + \sum_{j=1}^{t-1} a_j X^j$, outputs shares $u_i = f(i)$ and commitments $W_j = H(a_j)$ where $a_0 = s$.
- $\{0, 1\} \leftarrow \text{Vf}(i, u_i, \{W_j\}_{j \in [0, t-1]})$: Outputs 1 iff share u_i is consistent with commitments.
- $s \leftarrow \text{Rec}(\{(i, u_i)\}_{i \in S})$: For $|S| \geq t$, computes $\mu_i = \prod_{j \in S \setminus \{i\}} \frac{j}{j-i}$ and outputs $s = \sum_{i \in S} \mu_i \cdot u_i$.
- $\delta_i \leftarrow \text{PDec}(u_i, c)$: Outputs partial decryption $\delta_i = c_1 \cdot u_i$.
- $m \leftarrow \text{Comb}(\{(i, \delta_i)\}_{i \in S}, c, t)$: Computes μ_i , then $\tilde{c}_1 = \sum_{i \in S} \mu_i \cdot \delta_i$, outputs $m = \lfloor p(c_0 + \tilde{c}_1)/q \rfloor \bmod p$.

3 VoteSentinel

We present VoteSentinel, a lattice-based voting scheme combining ML-KEM for secure trustee communication, BFV homomorphic encryption for privacy-preserving tallying, and verifiable secret sharing for distributed trust, achieving coercion resistance through deterministic credential-based re-voting.

3.1 Scheme Model and Parties

The Election Authority (EA) initializes the election, manages registration, and maintains the bulletin board. Trustees $T_1, \dots, T_n$ collectively manage keys via threshold cryptography; this threshold structure provides key-loss resilience, meaning if up to $n-t$ trustees lose keys, the remaining t can complete the election. Voters $V_1, \dots, V_N$ receive credentials during registration and use them to construct ballots. The Bulletin Board BB is a public, append-only structure recording all election data. The Judge J evaluates evidence to determine accountability in disputes; decisions are publicly verifiable since all inputs are public or verifiable against public commitments.

3.2 System Specification and Parameters

The polynomial ring $R_q = \mathbb{Z}_q[X]/(X^d + 1)$ provides the algebraic structure for BFV encryption, where the degree d is a power of 2 (e.g., $d = 2048$ or $d = 4096$). The ciphertext modulus q is a large prime determining ciphertext coefficient size, and the plaintext modulus p defines plaintext space $R_p = \mathbb{Z}_p[X]/(X^d + 1)$. The implicit secret key $\text{sk} = \sum_{i \in \mathcal{Q}} s_i$ is unknown to any single party. Nullifier resolution NullifierResolve in VS ensures only the ballot with the highest counter per voter is counted (strong non-reusability [14]).

3.3 Vote Encoding Methods

We use One-hot [63], Coefficient Encoding [37], and bit-vector [64]/bit-packing [65] techniques for encoding single-choice, ranked-choice, and identifier, respectively.

Single-Choice Encoding [63]. For k candidates, define $\text{EncS} : [k] \rightarrow R_p$ by $\text{EncS}(v) = X^{v-1}$ and $\text{DecS} : R_p \rightarrow \mathbb{Z}^k$ extracting coefficients. To validate, all we need to do is to calculate $\sum_{i=0}^{k-1} a_i = 1$ per ballot.

Ranked-Choice Encoding [37]. For k candidates with ranking $\pi = (\pi_1, \dots, \pi_k)$ where $\pi_i \in [0, k-1]$ is points for candidate i , define $\text{EncR} : [0, k-1]^k \rightarrow R_p$ by $\text{EncR}(\pi) = \sum_{i=1}^k \pi_i \cdot X^{i-1}$. Validity: $\sum_{i=0}^{k-1} a_i = k(k-1)/2$ per ballot.

Identifier Encoding [65]. For nullifier base $\nu \in \{0, 1\}^\lambda$ and counter $\ell \in [0, L)$, define $\text{EncID} : \{0, 1\}^\lambda \times [0, L) \rightarrow R_p$ by $\text{EncID}(\nu, \ell) = \sum_{i=0}^{\lambda-1} \nu[i] \cdot X^i + \ell \cdot X^\lambda$ where $\nu[i]$ is the i -th bit of ν .

3.4 VoteSentinel Procedures

Definition 6 (VoteSentinel VS). Let N be voter count, n trustee count, $t = \lceil (n+1)/2 \rceil$ the threshold, k candidate count, L maximum votes per voter, and $\text{mode} \in \{\text{single}, \text{ranked}\}$. The voting system $\text{VS} = (\text{Setup}, \text{Reg}, \text{Vote}, \text{Tally}, \text{Vf}, \text{Judge})$ based on Key Encapsulation ML-KEM = (KG, Enc, Dec), Fully Homomorphic Encryption Scheme BFV = (KG, Enc, Dec, Add), and Verifiable Secret Sharing VSS = (Shr, Vf, Rec, PDec, Comb) is defined in Figures 1, 2, and 3.

The EA generates BFV parameters (d, q, p) with $d > \lambda + \lceil \log_2 L \rceil$ and $p > N \cdot k^2$, samples $a \leftarrow \$R_q$, and initializes H . Each trustee T_i generates ML-KEM keys $(\text{ek}_i, \text{dk}_i) \leftarrow \text{ML-KEM.KG}(1^\lambda)$, samples $\alpha_i \leftarrow \$R_q$, and runs $\text{VSS.Shr}(\alpha_i, t, n)$ to obtain shares $\{u_{i,j}\}_{j \in [n]}$ and commitments $\{W_{i,\ell}\}_{\ell \in [0, t-1]}$ published on BB. Shares

are distributed via ML-KEM: T_i sends $(\psi_{i,j}, \sigma_{i,j} = u_{i,j} \oplus K_{i,j})$ to T_j where $(K_{i,j}, \psi_{i,j}) \leftarrow \text{ML-KEM.Enc}(\text{ek}_j)$. Each T_j recovers $\hat{u}_{i,j}$, verifies against $\{W_{i,\ell}\}_\ell$, posts complaints for failures, and computes $s_j = \sum_{i \in \mathcal{Q}} \hat{u}_{i,j}$ where $\mathcal{Q}$ is the qualified trustee set. If $|\mathcal{Q}| \geq t$, each $T_i \in \mathcal{Q}$ publishes $b_i = -a \cdot s_i + \varepsilon_i$ (where $\varepsilon_i \leftarrow \chi_{\text{err}}$) and commitment $\Gamma_i = H(s_i \| \omega_i)$ for random salt ω_i . The public key is $\text{pk} = (a, \sum_{i \in \mathcal{Q}} b_i)$.

For eligible voter $j \in \mathcal{E}$ with $j \notin \mathcal{R}$, the EA samples $\xi_j \leftarrow \{0,1\}^{2\lambda}$, computes nullifier $\nu_j = H(\xi_j \| j \| 0x00)$, updates $\mathcal{R} \leftarrow \mathcal{R} \cup \{j\}$, and returns $\text{cred}_j = (\xi_j, \nu_j)$.

$\text{Setup}(1^\lambda, n, t, k, N, L, \text{mode})$	$\text{DistKeyGen}(n, t, \{\text{ek}_i, \text{dk}_i\}_{i \in [n]})$	$\text{PubKeyGen}(a, \mathcal{Q}, \{s_i\}_{i \in \mathcal{Q}})$
1: $(d, q, p) \leftarrow \text{Param}(1^\lambda)$ s.t. $d > \lambda + \lceil \log_2 L \rceil \wedge p > N \cdot k^2$ 2: $H \leftarrow \text{HashGen}(1^\lambda)$; $a \leftarrow \mathbb{Z}_{R_q}$; $\mathcal{E} \leftarrow \text{eligibleVoters}$ 3: $\{(\text{ek}_i, \text{dk}_i)\}_{i \in [n]} \leftarrow \text{TrusteeKeyGen}(n)$ 4: $(\mathcal{Q}, \{s_i\}_{i \in \mathcal{Q}}) \leftarrow \text{DistKeyGen}(n, t, \{\text{ek}_i, \text{dk}_i\}_{i \in [n]})$ 5: if $ \mathcal{Q} < t$ then return $\perp$ 6: $\text{pk} \leftarrow \text{PubKeyGen}(a, \mathcal{Q}, \{s_i\}_{i \in \mathcal{Q}})$ 7: $\{\Gamma_i\}_{i \in \mathcal{Q}}, \{\omega_i\}_{i \in \mathcal{Q}} \leftarrow \text{CommitKeys}(\mathcal{Q}, \{s_i\}_{i \in \mathcal{Q}}, H)$ 8: $\text{pp} \leftarrow (d, q, p, t, k, N, L, \text{mode}, H, \text{pk}, \{\Gamma_i\}_{i \in \mathcal{Q}}, \mathcal{Q}, \mathcal{E})$ 9: return $(\text{pp}, \{\{s_i, \omega_i\}\}_{i \in \mathcal{Q}})$	1: for $i \in [n]$ do 2: $\alpha_i \leftarrow \mathbb{Z}_{R_q}$ 3: $\{(\{u_{i,j}\}_{j \in [n]}, \{W_{i,\ell}\}_{\ell \in [0,t-1]}) \leftarrow \text{VSS.Shr}(\alpha_i, t, n)$ 4: $\text{BB} \leftarrow \text{BB} \cup \{(\text{com-vss}, i, \{W_{i,\ell}\}_\ell)\}$ 5: for $i \in [n]$ do 6: for $j \in [n] \setminus \{i\}$ do 7: $(K_{i,j}, \psi_{i,j}) \leftarrow \text{ML-KEM.Enc}(\text{ek}_j)$ 8: $\sigma_{i,j} \leftarrow u_{i,j} \oplus K_{i,j}$ 9: $\text{Send}(T_i \rightarrow T_j : (\psi_{i,j}, \sigma_{i,j}))$ 10: $\mathcal{Q} \leftarrow [n]$ 11: for $j \in [n]$ do 12: for $i \in [n] \setminus \{j\}$ do 13: $K_{i,j} \leftarrow \text{ML-KEM.Dec}(\text{dk}_j, \psi_{i,j})$ 14: $\hat{u}_{i,j} \leftarrow \sigma_{i,j} \oplus K_{i,j}$ 15: if $\text{VSS.Vf}(j, \hat{u}_{i,j}, \{W_{i,\ell}\}_\ell) = 0$ then 16: $\text{BB} \leftarrow \text{BB} \cup \{(\text{complain}, j, i, \hat{u}_{i,j})\}$ 17: $\mathcal{Q} \leftarrow \mathcal{Q} \setminus \{i\}$ 18: $s_j \leftarrow u_{j,j} + \sum_{i \in \mathcal{Q} \setminus \{j\}} \hat{u}_{i,j}$ 19: return $(\mathcal{Q}, \{s_i\}_{i \in \mathcal{Q}})$	1: for $i \in \mathcal{Q}$ do 2: $\varepsilon_i \leftarrow \mathbb{Z}_{\chi_{\text{err}}}$; $b_i \leftarrow -a \cdot s_i + \varepsilon_i$ 3: $\text{BB} \leftarrow \text{BB} \cup \{(\text{pk-share}, i, b_i)\}$ 4: $b \leftarrow \sum_{i \in \mathcal{Q}} b_i$ 5: return (a, b)
$\text{CommitKeys}(\mathcal{Q}, \{s_i\}_{i \in \mathcal{Q}}, H)$	$\text{Reg}(\text{pp}, j)$	
1: for $i \in \mathcal{Q}$ do 2: $\omega_i \leftarrow \{0,1\}^\lambda$; $\Gamma_i \leftarrow H(s_i \ \omega_i)$ 3: $\text{BB} \leftarrow \text{BB} \cup \{(\text{com-key}, i, \Gamma_i)\}$ 4: return $(\{\Gamma_i\}_{i \in \mathcal{Q}}, \{\omega_i\}_{i \in \mathcal{Q}})$	1: if $j \notin \mathcal{E} \vee j \in \mathcal{R}$ then return $\perp$ 2: $\xi_j \leftarrow \{0,1\}^{2\lambda}$ 3: $\nu_j \leftarrow H(\xi_j \ j \ 0x00)$ 4: $\mathcal{R} \leftarrow \mathcal{R} \cup \{j\}$; $\text{cred}_j \leftarrow (\xi_j, \nu_j)$ 5: return cred_j	

Fig. 1: The setup and registration phases in VoteSentinel

To cast vote v with re-voting attempt sequence counter $\ell \in [0, L)$, voter V_j derives all randomness from credential ξ_j via $\rho_1, \rho_2, r \leftarrow H(\xi_j \| \ell \| \cdot)$ -this determinism is important for individual verifiability (voters can recompute their ballots) and coercion resistance (voters can construct fake credentials producing different ballots). The vote is encoded as $m = \text{EncS}(v)$ or $m = \text{EncR}(v)$ depending on voting mode and encrypted as $c \leftarrow \text{BFV.Enc}(\text{pk}, m; \rho_1)$, hiding the choice under RLWE. To enable nullifier resolution during tallying, the voter also encrypts their identifier $m' = \text{EncID}(\nu_j, \ell)$ as $c' \leftarrow \text{BFV.Enc}(\text{pk}, m'; \rho_2)$, packaging both nullifier and counter together. The commitment $\gamma = H(c \| c' \| \nu_j \| \ell \| r)$ binds the voter to this ballot, enabling dispute resolution: if the ballot is dropped, the voter can open γ to prove they submitted it. The complete ballot $\beta = (c, c', \gamma)$ is posted to BB.

Given participating trustees $S \subseteq \mathcal{Q}$ with $|S| \geq t$, tallying proceeds in four stages. First, all ballots are collected from the bulletin board as $\mathcal{B} \leftarrow \{(c, c', \gamma) : (\text{ballot}, (c, c', \gamma)) \in \text{BB}\}$, ensuring only properly formatted ballots are considered. Second, trustees jointly decrypt each identifier ciphertext c' via partial decryptions $\delta'_i = c'_i \cdot s_i$ combined with Lagrange interpolation to recover $(\nu_{c'}, \ell_{c'})$ this reveals which ballots supersede others without revealing vote contents. Third, nullifier resolution implements strong non-reusability: for each nullifier ν , only the ballot with highest counter ℓ is retained in final set $\mathcal{F}$, enabling re-voting while preventing double-counting. Fourth, valid vote ciphertexts are aggregated homomorphically

as $C = \sum_{c \in \mathcal{F}} c$ computed via repeated application of BFV.Add leveraging BFV's additivity property, this homomorphic aggregation computes the encrypted tally $\text{BFV.Enc}(\text{pk}, \sum_{c \in \mathcal{F}} m_c)$ without learning individual votes. Each trustee T_i then publishes partial decryption $\delta_i = C_1 \cdot s_i$ with proof $\pi_i = H(\delta_i \| \Gamma_i)$ binding it to their committed key share. This proof enables accountability if a trustee provides malformed decryption shares. Combining shares via VSS.Comb yields plaintext result M , decoded as vote tallies $(r_0, \dots, r_{k-1})$.

$\text{Vote}(\text{pp}, \text{cred}_j, v, \ell)$	$\text{Tally}(\text{pp}, \text{BB}, \{(s_i, \omega_i)\}_{i \in S})$	$\text{ThresholdDecrypt}(\mathcal{F}, S, \{s_i\}_{i \in S}, \{\omega_i\}_{i \in S}, t)$
1: $(\xi_j, \nu_j) \leftarrow \text{cred}_j$ 2: if $\text{mode} = \text{single} \wedge v \notin [k]$ then return $\perp$ 3: if $\text{mode} = \text{ranked} \wedge v \notin [0, k-1]^k$ then return $\perp$ 4: if $\ell \notin [0, L]$ then return $\perp$ 5: $\rho_1 \leftarrow H(\xi_j \ \ell \ 0x01)$ 6: $\rho_2 \leftarrow H(\xi_j \ \ell \ 0x02)$ 7: $r \leftarrow H(\xi_j \ \ell \ 0x03)$ 8: if $\text{mode} = \text{single}$ then $m \leftarrow \text{EncS}(v)$ 9: else $m \leftarrow \text{EncR}(v)$ 10: $c \leftarrow \text{BFV.Enc}(\text{pk}, m; \rho_1)$ 11: $m' \leftarrow \text{EncID}(\nu_j, \ell)$ 12: $c' \leftarrow \text{BFV.Enc}(\text{pk}, m'; \rho_2)$ 13: $\gamma \leftarrow \text{Com.Cmt}(c \ c' \ \nu_j \ \ell; r)$ 14: $\beta \leftarrow (c, c', \gamma)$ 15: $\text{BB} \leftarrow \text{BB} \cup \{(\text{ballot}, \beta)\}$ 16: return (β, r)	1: if $ S < t$ then return $\perp$ 2: $\mathcal{B} \leftarrow \{(c, c', \gamma) : (\text{ballot}, (c, c', \gamma)) \in \text{BB}\}$ 3: $\mathcal{F} \leftarrow \text{NullifierResolve}(\mathcal{B}, S, \{s_i\}_{i \in S}, t)$ 4: $\text{BB} \leftarrow \text{BB} \cup \{(\text{final-set}, \mathcal{F})\}$ 5: if $ \mathcal{F} > N$ then return $(\perp, \text{stuffing})$ 6: $(\text{res}, \{\delta_i\}_{i \in S}) \leftarrow \text{ThresholdDecrypt}(\mathcal{F}, S, \{s_i\}_{i \in S}, \{\omega_i\}_{i \in S}, t)$ 7: $\text{BB} \leftarrow \text{BB} \cup \{(\text{result}, \text{res})\}$ 8: return $(\text{res}, \mathcal{F}, \{\delta_i\}_{i \in S})$ $\text{NullifierResolve}(\mathcal{B}, S, \{s_i\}_{i \in S}, t)$ 1: for $(c, c', \gamma) \in \mathcal{B}$ do 2: for $i \in S$ do 3: $\delta'_i \leftarrow \text{VSS.PDec}(s_i, c')$ 4: $m'_{\nu'} \leftarrow \text{VSS.Comb}(\{(i, \delta'_i)\}_{i \in S}, c', t)$ 5: $(\nu_{\nu'}, \ell_{\nu'}) \leftarrow \text{DecID}(m'_{\nu'})$ 6: $\mathcal{M} \leftarrow \emptyset$ 7: for $(c, c', \gamma) \in \mathcal{B}$ do 8: if $\nu_{\nu'} \notin \text{dom}(\mathcal{M}) \vee \mathcal{M}[\nu_{\nu'}].\ell < \ell_{\nu'}$ then 9: $\mathcal{M}[\nu_{\nu'}] \leftarrow (c, \ell_{\nu'})$ 10: $\mathcal{F} \leftarrow \{c : \exists \nu, (c, \cdot) = \mathcal{M}[\nu]\}$ 11: return $\mathcal{F}$	1: $C \leftarrow (0, 0)$ 2: for $c \in \mathcal{F}$ do $C \leftarrow \text{BFV.Add}(C, c)$ 3: $\text{BB} \leftarrow \text{BB} \cup \{(\text{aggregate}, C)\}$ 4: for $i \in S$ do 5: $\delta_i \leftarrow \text{VSS.PDec}(s_i, C)$ 6: $\pi_i \leftarrow H(\delta_i \ \Gamma_i)$ 7: $\text{BB} \leftarrow \text{BB} \cup \{(\text{pdec}, i, \delta_i, \pi_i)\}$ 8: $M \leftarrow \text{VSS.Comb}(\{(i, \delta_i)\}_{i \in S}, C, t)$ 9: if $\text{mode} = \text{single}$ then 10: $(r_0, \dots, r_{k-1}) \leftarrow \text{DecS}(M)$ 11: if $\sum_{i=0}^{k-1} r_i \neq \mathcal{F} $ then return $(\perp, \text{invalid})$ 12: else 13: $(r_0, \dots, r_{k-1}) \leftarrow \text{DecR}(M)$ 14: if $\sum_{i=0}^{k-1} r_i \neq \mathcal{F} \cdot k(k-1)/2$ then return $(\perp, \text{invalid})$ 15: $\text{res} \leftarrow (r_0, \dots, r_{k-1})$ 16: return $(\text{res}, \{\delta_i\}_{i \in S})$

Fig. 2: The voting and tally procedures in VoteSentinel

Individual: Voter recomputes (c, c', γ) from $(\text{cred}_j, v, \ell, r)$ and checks presence on BB and inclusion in $\mathcal{F}$. *Universal:* Observer verifies $|\mathcal{F}| \leq N$, recomputes C from $\mathcal{F}$, checks proofs $\pi_i = H(\delta_i \| \Gamma_i)$, and validates $\sum_i r_i$ against expected totals.

When verification fails or parties misbehave, the accountability mechanism activates: the judge Jdg processes cryptographic evidence E to identify and blame the responsible party, ensuring no honest party is falsely accused. Evidence types correspond to specific protocol violations: **drop** captures EA censorship (voter opens commitment γ to prove ballot submission, yet ballot absent from BB), **not-counted** captures trustee manipulation during nullifier resolution (voter proves their ballot has highest counter for nullifier ν yet excluded from final set $\mathcal{F}$), **bad-pdec** captures malicious threshold decryption (trustee T_i opens commitment Γ_i to reveal key share s_i , enabling recomputation of correct partial decryption and exposing mismatch with published δ_i), and **bad-share** captures faulty setup (trustee proves received share $\hat{u}_{i,j}$ fails VSS verification against commitments $\{W_{i,\ell}\}_\ell$). All evidence is publicly verifiable against commitments on BB, ensuring the judge's blame assignment is transparent and non-repudiable. This implements accountability (Definition 15) by guaranteeing that any deviation from honest execution produces verifiable evidence against the misbehaving party, while honest parties can never be blamed.

$Vf(pp, BB, mode, aux)$	$Judge(pp, BB, E)$
1: if mode = ind then	1: if $pk \notin R_c^2$ then return (EA, err ₁)
2: $(cred_j, v, \ell, r) \leftarrow aux$	2: $B \leftarrow \{ (ballot, \cdot) \in BB \}$
3: $(\xi_j, \nu_j) \leftarrow cred_j$	3: if $ B > N \cdot L$ then return (EA, err ₂)
4: $\rho_1 \leftarrow H(\xi_j \ \ell \ 0x01)$	4: $(final_set, \mathcal{F}) \leftarrow BB$
5: $\rho_2 \leftarrow H(\xi_j \ \ell \ 0x02)$	5: if $ \mathcal{F} > N$ then return (Trustees, err ₃)
6: if pp.mode = single then $m \leftarrow EncS(v)$	6: for (type, data) $\in E$ do
7: else $m \leftarrow EncR(v)$	7: if type = drop then
8: $c \leftarrow BFV.Enc(pk, m; \rho_1)$	8: $(j, c, c', \gamma, \nu_j, \ell, r) \leftarrow data$
9: $m' \leftarrow EncID(\nu_j, \ell)$	9: if $Com.Opn(\gamma, c \ c' \ \nu_j \ \ell, r) = 1$ then
10: $c' \leftarrow BFV.Enc(pk, m'; \rho_2)$	10: if $(ballot, (c, c', \gamma)) \notin BB$ then
11: $\gamma \leftarrow Com.Cmt(c \ c' \ \nu_j \ \ell; r)$	11: return (EA, err ₄ , j)
12: return $(ballot, (c, c', \gamma)) \in BB \wedge c \in \mathcal{F}$	12: if type = not-counted then
13: if mode = univ then	13: $(j, c, c', \nu_j, \ell) \leftarrow data$
14: $(res, C', \mathcal{F}, \{\delta_i\}_{i \in S}) \leftarrow aux$	14: if $(ballot, (c, c', \gamma)) \in BB \wedge c \notin \mathcal{F}$ then
15: if $ \mathcal{F} > N$ then return 0	15: $m' \leftarrow EncID(\nu_j, \ell)$
16: $C' \leftarrow (0, 0)$	16: if $\exists \ell' > \ell: EncID(\nu_j, \ell')$ in any $c'' \in BB$ then
17: for $c \in \mathcal{F}$ do $C' \leftarrow BFV.Add(C', c)$	17: return (Trustees, err ₅ , j)
18: if $C' \neq C$ then return 0	18: if type = bad-pdec then
19: for $i \in S$ do	19: $(i, \delta_i, s_i, \omega_i) \leftarrow data$
20: if $(pdec, i, \delta_i, \pi_i) \in BB \wedge \pi_i \neq H(\delta_i \ I_i)$ then	20: if $H(s_i \ \omega_i) = I_i$ then
21: return 0	21: $(aggregate, C') \leftarrow BB$
22: $(r_0, \dots, r_{k-1}) \leftarrow res$	22: $\delta'_i \leftarrow VSS.PDec(s_i, C')$
23: if pp.mode = single $\wedge \sum_i r_i \neq \mathcal{F} $ then return 0	23: if $(pdec, i, \delta_i, \cdot) \in BB \wedge \delta'_i \neq \delta_i$ then
24: if pp.mode = ranked $\wedge \sum_i r_i \neq \mathcal{F} \cdot k(k-1)/2$ then return 0	24: return (T _i , err ₆)
25: return 1	25: if type = bad-share then
	26: $(i, j, \hat{u}_{i,j}, \{W_{i,\ell}\}_\ell) \leftarrow data$
	27: if $(com-vss, i, \{W_{i,\ell}\}_\ell) \in BB \wedge VSS.Vf(j, \hat{u}_{i,j}, \{W_{i,\ell}\}_\ell) = 0$ then
	28: return (T _i , err ₇)
	29: return $\perp$

Fig. 3: The verification and dispute resolution functions in VoteSentinel

4 Privacy and Verifiability Analysis of VoteSentinel

Here, we provide intuition, and due to space limitations, add the complete proofs here: https://anonymous.4open.science/r/VoteSentinel-FF53/Anonymous_Manuscript.pdf. For example, Ideal Functionality of VoteSentinel is touchable as Ballot Privacy + (Strong Consistency $\wedge$ Correctness) $\stackrel{[19]}{\Rightarrow}$ Ideal Functionality.

To address the key-loss resiliency, in the IACR 2025 election failure [21, 46, 47], Helios's (3,3)-threshold caused irrecoverable failure when one trustee lost their key. Under VoteSentinel with $(t=2, n=3)$ -threshold, when trustee T_3 loses s_3 , the remaining trustees $S = \{1, 2\}$ satisfy $|S| \geq t$, enabling VSS.Comb to recover sk via Lagrange interpolation: $\tilde{s} = \mu_1 s_1 + \mu_2 s_2$ where $\mu_1 = 2, \mu_2 = -1$, yielding $\tilde{s} = 2s_1 - s_2 = sk$, thereby completing threshold decryption through $\delta_i = VSS.PDec(s_i, C')$ for $i \in S$ (cf. Tally, Section 3), guaranteeing election completion despite loss of up to $n-t=1$ trustee keys under (t, n) -VSS correctness (Definition 5).

4.1 Resistance to Coercion

The Coercion Resistance property implies many other privacy guarantees; i.e., Coercion Resistance [12] $\stackrel{[20]}{\Rightarrow}$ Receipt-freeness [10] $\stackrel{[20]}{\Rightarrow}$ Privacy with Malicious Board [9] $\stackrel{[9]}{\Rightarrow}$ Ballot Privacy [19] $\stackrel{[66]}{\Rightarrow}$ Individual Verifiability [11] $\stackrel{[18]}{\Leftarrow}$ Accountability [17]. Among all definitions of coercion resistance, Juels, Catalano & Jakobsson (JCJ) has been well-studied, and we consider the patched version of it [12] as follows.

Definition 7 (Coercion Resistance [12]). Let $\Gamma = (\text{Setup}, \text{Register}, \text{Vote}, \text{Tally})$ be an election scheme, n_a , n_c and n_v be integers, and $\mathcal{D}$ be a distribution over $\{1, \dots, n_c, \phi, \lambda\}^{n_v}$. We say Γ satisfies JCJ with respect to n_a , n_c , n_v , $\mathcal{D}$, if for all probabilistic polynomial-time adversaries $\mathcal{A}$, there exists probabilistic polynomial-time algorithm $\mathcal{B}$, algorithm `fakecred`, and a negligible function negl , such that for all security parameters κ , we have:

$|\Pr[\text{JCJ}(\Gamma, \mathcal{A}, n_a, n_c, n_v, \text{fakecred}, \mathcal{D}, \kappa) = 1] - \Pr[\text{JCJ-}\$(\Gamma, \mathcal{B}, n_a, n_c, n_v, \mathcal{D}, \kappa) = 1]| \leq \text{negl}(\kappa)$ where games JCJ and JCJ-\$ are defined as follows in Figure 4b, and algorithm `fakecred` takes a public key, a public credential, and a private credential as input, and outputs a (fake) private credential.

The JCJ-\$\$(\Gamma, \mathcal{B}, n_a, n_c, n_v, \mathcal{D}, \kappa) game is identical to JCJ [12] except:

- Line 13: $bb_2 \leftarrow \mathcal{B}(sd', M, |bb_1|)$ (receives bulletin board length)
- Line 14: $v \leftarrow \text{Ideal-Tally}(sk, bb_1 \cup bb_2, L, n_c, \kappa)$
- Line 15: $g \leftarrow \mathcal{B}(v)$ (no proof)

where `Ideal-Tally` tallies bb_1 normally and specially handles $bb_2 \setminus bb_1$ by counting ballots from corrupt voters without double-voting.

Theorem 1. Let H be modeled as a quantum random oracle. Assuming $\text{MLWE}_{n,q,\chi}$ and $\text{RLWE}_{n,q,\chi}$ problems are hard for QPT adversaries, VS satisfies coercion resistance (Definition 7). Specifically, there exists a QPT simulator `fakecred` such that for all QPT adversaries $\mathcal{A}$, there exists QPT $\mathcal{B}$ with:

$$|\Pr[\text{JCJ}(\text{VS}, \mathcal{A}, n_a, n_c, n_v, \text{fakecred}, \mathcal{D}, \lambda) = 1] - \Pr[\text{JCJ-}\$(\text{VS}, \mathcal{B}, n_a, n_c, n_v, \mathcal{D}, \lambda) = 1]| \leq \text{negl}(\lambda)$$

The `fakecred` algorithm generates indistinguishable fake credentials by exploiting the re-voting mechanism: a coerced voter submits a compliant ballot with counter ℓ , then later submits their true vote with counter $\ell' > \ell$. The nullifier resolution in `Tally` ensures only the highest-counter ballot is counted, while IND-CPA security of BFV prevents the coercer from distinguishing real from fake credentials.

4.2 Participation Privacy

Definition 8 ((δ, k_{priv})-Participation Privacy [10]). A voting scheme $\mathcal{S}$ achieves (δ, k)-participation privacy if for any PPT adversary $\mathcal{A}$, any two honest voters id_0, id_1 , and any $k' \leq k$, then $|\Pr[\text{Exp}_{\mathcal{A}, \mathcal{S}, k}^{\text{priv}, 0} = 0] - \Pr[\text{Exp}_{\mathcal{A}, \mathcal{S}, k}^{\text{priv}, 1} = 0]| \leq \delta$, where the experiment $\text{Exp}_{\mathcal{A}, \mathcal{S}, k}^{\text{priv}, \beta}$ is defined as Setup phase:

1. Run setup to obtain public parameters and keys
2. Initialize two bulletin boards $\text{BB}_0, \text{BB}_1 \leftarrow \emptyset$
3. Register eligible voters including id_0, id_1
4. Choose $\beta \leftarrow_R \{0, 1\}$

In the query phase, the adversary $\mathcal{A}$ has access to a set of the oracles $\mathcal{Q}_S = \{\text{OCast}, \text{OVoteAbstain}, \text{OVoteLR}, \text{OTally}\}$ and sees BB_β . Eventually, $\mathcal{A}$ outputs guess β' . The experiment returns $(\beta' = \beta)$. The scheme achieves (δ, k)-participation privacy if no adversary can distinguish whether voter id_0 abstained while id_1 voted up to k times, or vice versa, with advantage greater than δ .

<pre> OVoteAbstain($v_1, \dots, v_k$) 1: if $k' > k$ then return $\perp$ 2: $b_{0,1}, \dots, b_{0,m_0} \leftarrow \text{VoteDummy}(id_0)$ 3: $b_{1,1}, \dots, b_{1,m_1} \leftarrow \text{VoteDummy}(id_1)$ 4: for $j = 1, \dots, k'$ do 5: $t'_j \leftarrow \mathbb{P}_1$ 6: $b'_{0,j} \leftarrow \text{Vote}'((id_\beta, sk_{id_0}), id_0, v_j, t'_j)$ 7: $b'_{1,j} \leftarrow \text{Vote}'((id_\beta, sk_{id_1}), id_1, v_j, t'_j)$ 8: endfor 9: Append($b_{0,1}, \dots, b_{0,m_0}, BB_0$) 10: Append($b_{1,1}, \dots, b_{1,m_1}, BB_1$) 11: Append($b'_{0,1}, \dots, b'_{0,k'}, BB_0$) 12: Append($b'_{1,1}, \dots, b'_{1,k'}, BB_1$) OCast($b$) 1: if Valid($BB_\beta, b$) then 2: Append($b, BB_\beta$) 3: Append($b, BB_1$) 4: endif OVoteLR(id, v_0, v_1, t) 1: $b_0 \leftarrow \text{Vote}'((id_\beta, sk_{id}), id, v_0, t)$ 2: $b_1 \leftarrow \text{Vote}'((id_\beta, sk_{id}), id, v_1, t)$ 3: if Valid(BB_β, b_β) = 0 then 4: return $\perp$ 5: endif 6: Append(b_0, BB_0); Append(b_1, BB_1) OTally() 1: if $\beta = 0$ then return Tally(sk, BB_0) 2: else 3: (R, II) $\leftarrow$ Tally(sk, BB_0) 4: $II' \leftarrow \text{SimTally}(BB_1, R)$ 5: endif 6: return (R, II') </pre>	<pre> Game JCJ($\Gamma, \mathcal{A}, n_a, n_c, n_v, \text{fakecred}, \mathcal{D}, \kappa$) 1: ($pk, sk, mb, mc$) $\leftarrow$ Setup(κ) 2: $V \leftarrow \mathcal{A}(pk, \kappa)$ 3: for $i \in [1, n_v]$ do 4: (pd_i, sd_i) $\leftarrow$ Register(pk, κ) 5: $L \leftarrow \{pd_1, \dots, pd_{n_v}\}$ 6: $M \leftarrow \{(i, sd_i) : i \in V \wedge i \in [1, n_v]\}$ 7: (j, v) $\leftarrow \mathcal{A}(M, L)$ 8: if $V \neq n_a \vee j \notin [1, n_v] \vee v \notin [1, n_c] \cup \{\phi\}$ then 9: return 0 10: if $\beta = 0$ then 11: if $v \neq \phi$ then 12: $b \leftarrow \text{Vote}(sd_j, pk, v, n_c, \kappa)$ 13: $bb_1 \leftarrow bb_1 \cup \{b\}$ 14: $sd' \leftarrow \text{fakecred}(pk, pd_j, sd_j)$ 15: else 16: $sd' \leftarrow sd_j$ 17: for $i \in [1, n_v] \setminus (\{j\} \cup V)$ do 18: $v \leftarrow \mathcal{D}$ 19: if $v \neq \phi$ then 20: if $v = \lambda$ then 21: $v \leftarrow \mathcal{S}[1, n_c]$ 22: $sd_i \leftarrow \text{fakecred}(pk, pd_i, sd_i)$ 23: $b \leftarrow \text{Vote}(sd_i, pk, v, n_c, \kappa)$ 24: $bb_1 \leftarrow bb_1 \cup \{b\}$ 25: else 26: $bb_2 \leftarrow \mathcal{A}(sd', bb_1)$ 27: (v, pf) $\leftarrow$ Tally($sk, bb_1 \cup bb_2, L, n_c, \kappa$) 28: $g \leftarrow \mathcal{A}(v, pf)$ 29: return ($\beta = g$) </pre>
--	--

(a) The Participation Privacy Oracles (b) The JCJ Game for Coercion Resistance

Fig. 4: Security Games for Coercion Resistance and Participation Privacy

Theorem 2. *Let H be modeled as $QROM$. Assuming $RLWE_{n,q,\chi}$ problem is hard for QPT adversaries, the voting scheme VS achieves (δ, L) -participation privacy (Definition 8) with $\delta \leq \text{negl}(\lambda)$. For all QPT adversaries $\mathcal{A}$, any two honest voters $id_0, id_1 \in \mathcal{E}$, and $k' \leq L$: $\left| \Pr[\text{Exp}_{\mathcal{A}, VS, L}^{\text{ppriv}, 0}(\lambda) = 1] - \Pr[\text{Exp}_{\mathcal{A}, VS, L}^{\text{ppriv}, 1}(\lambda) = 1] \right| \leq \text{negl}(\lambda)$.*

Voter identities in VS are encoded in encrypted identifier ciphertexts $c' = \text{BFV.Enc}(pk, \text{EncID}(\nu_j, \ell))$, which are IND-CPA secure under $RLWE$. The nullifiers $\nu_j = H(\xi_j \| j \| 0x00)$ are computationally hidden, and the public BB reveals only ciphertexts, not participation patterns.

4.3 Verifiability

It has been shown that privacy is not achievable without individual verifiability [66]. But we do further check on VS by Theorem 3 that states VS satisfies both qualitative and quantitative verifiability under the standard and well-studied framework of Küsters et al. [11], demonstrating that honest voters who perform verification checks can detect any manipulation of their ballots with overwhelming

⁴ for each voter either a vote choice (1 to n_c), abstention (ϕ), or random vote with fake credential (λ)

probability, while the bulletin board integrity and threshold decryption mechanism ensure universal verifiability of the tallying process.

Definition 9 (Verifiability [11]). Let $\mathcal{P}$ be a protocol with the set of agents Σ . Let $\delta \in [0,1]$ be the tolerance, $J \in \Sigma$ be the judge and γ be a goal. Then, the protocol $\mathcal{P}$ is (γ, δ) -verifiable by the judge J if for all adversaries π_A and $\pi = (\pi_{\mathcal{P}} \parallel \pi_A)$, the probability $\Pr[\pi(\ell) \rightarrow \neg\gamma, (J : \text{accept})]$ is δ -bounded as a function of ℓ .

The following Definition 10 describes the qualitative verifiability goal, and Definition 11 recalls the quantitative verifiability goal from [11].

Definition 10. The qualitative goal $\gamma_{ql}(\varphi)$ is satisfied in a run r if either (a) the trust assumption φ does not hold true in r , or if (b) φ holds true in r and there exist valid choices $\tilde{c}_1, \dots, \tilde{c}_n$ for which the following conditions are satisfied:

1. An election result is published in r and it is equal to $\rho(\tilde{c}_1, \dots, \tilde{c}_n)$.
2. The multiset $\{\tilde{c}_1, \dots, \tilde{c}_n\}$ consists of:
 - all the honest voters' actual choices who successfully performed their check,
 - a subset of actual choices of honest voters who did not perform their check (successfully),
 - at most n_d additional choices (where n_d is the number of dishonest voters).

Definition 11. Let d be distance function defined by $d(\mathbf{c}, \mathbf{c}') = \sum_{c \in \mathcal{C}} |f_{\text{count}}(\mathbf{c})[c] - f_{\text{count}}(\mathbf{c}')[c]|$ where f_{count} counts occurrences of each choice. The quantitative goal $\gamma_{qn}(k, \varphi)$ is satisfied in a run r if either (a) the trust assumption φ does not hold true in r , or if (b) φ holds true in r and there exist valid choices $c'_1, \dots, c'_{n_d}$ (representing possible choices of dishonest voters) and $\tilde{c}_1, \dots, \tilde{c}_n$, such that:

1. An election result is published and it is equal to $\rho(\tilde{c}_1, \dots, \tilde{c}_n)$, and
2. $d((c_1, \dots, c_{n_h}, c'_1, \dots, c'_{n_d}), (\tilde{c}_1, \dots, \tilde{c}_n)) \leq k$.

where $c_1, \dots, c_{n_h}$ are the actual choices of honest voters.

Theorem 3. Let φ_t denote the trust assumption that at least t trustees are honest and BB is append-only. The voting scheme VS is $(\gamma_{ql}(\varphi_t), 0)$ -verifiable and $(\gamma_{qn}(0, \varphi_t), \delta)$ -verifiable with $\delta \leq \text{negl}(\lambda)$ (Definitions 9–11). Formally, for judge $J = \mathbf{Vf}$ and all adversaries π_A :

$$\Pr[\pi \rightarrow \neg\gamma_{ql}(\varphi_t), (J : \text{accept})] = 0 \quad \wedge \quad \Pr[\pi \rightarrow \neg\gamma_{qn}(0, \varphi_t), (J : \text{accept})] \leq \text{negl}(\lambda)$$

Individual verifiability follows from deterministic ballot construction: voters recompute (c, c', γ) from (cred_j, v, ℓ) and verify presence on BB. Universal verifiability holds because the homomorphic aggregation $C' = \sum_{c \in \mathcal{F}} c$ is publicly recomputable, partial decryptions δ_i are verified against commitments $T_i = H(s_i \parallel \omega_i)$, and the sum constraint $\sum_i r_i = |\mathcal{F}|$ detects any manipulation.

4.4 Everlasting Privacy

This concept is based on the principle that a voter's individual choice remains secret forever, even against an adversary with unlimited computational power in the future who could potentially break today's cryptographic assumptions. We use the definition [36], which is recommended by [13] as well.

Definition 12 (Practical Everlasting Privacy [36]). Let the voting scheme $\Pi = (\text{Setup}, \text{Register}, \text{Vote}, \text{Tally}, \text{Verify})$ with n voters $\mathcal{V} = \{v_1, \dots, v_n\}$, m trustees $\mathcal{T} = \{T_1, \dots, T_m\}$, and public bulletin board BB.

Consider the experiment $\text{Exp}_{\Pi, \mathcal{A}}^{\text{priv}}(\lambda, b)$:

1. $(\text{pp}, \text{sk}_{\text{sys}}, \text{state}_0) \leftarrow \text{Setup}(1^\lambda)$ where sk_{sys} denotes system secrets (trustee shares, decryption keys)
2. $(\text{state}_1, i_0, i_1, \text{vote}_0, \text{vote}_1) \leftarrow \mathcal{A}_1^{\mathcal{O}_{\text{reg}}, \mathcal{O}_{\text{vote}}}(\text{pp}, \text{state}_0)$ where:
 - $\mathcal{O}_{\text{reg}}$: Registration oracle allowing $\mathcal{A}$ to register honest voters
 - $\mathcal{O}_{\text{vote}}$: Voting oracle allowing $\mathcal{A}$ to cast votes for corrupted voters
 - $i_0, i_1 \in \mathcal{V}$ are two uncorrupted voters
 - $\text{vote}_0, \text{vote}_1$ are valid votes with $\rho(\dots, \text{vote}_0, \dots) = \rho(\dots, \text{vote}_1, \dots)$ for result function ρ
3. Execute **Vote** for voter i_b with vote vote_b , adding the resulting ballot to BB
4. $(\text{result}, \pi_{\text{tally}}) \leftarrow \text{Tally}(\text{pp}, \text{BB}, \text{sk}_{\text{sys}})$
5. $\text{view}_{\mathcal{A}} \leftarrow (\text{pp}, \text{BB}, \text{result}, \pi_{\text{tally}}, \text{state}_1)$
6. $b' \leftarrow \mathcal{A}_2^\infty(\text{view}_{\mathcal{A}})$ where $\mathcal{A}_2^\infty$ is computationally unbounded and output b' .

Protocol Π satisfies practical everlasting privacy if for all probabilistic polynomial-time adversaries $\mathcal{A}_1$ (modeling adversarial behavior during election) and all computationally unbounded adversaries $\mathcal{A}_2$ (modeling future adversary):

$$|\Pr[\text{Exp}_{\Pi, \mathcal{A}}^{\text{priv}}(\lambda, 0) = 1] - \Pr[\text{Exp}_{\Pi, \mathcal{A}}^{\text{priv}}(\lambda, 1) = 1]| \leq \text{negl}(\lambda)$$

Theorem 4. Let H be modeled as a quantum random oracle. The voting scheme VS satisfies practical everlasting privacy (Definition 12). For all QPT adversaries $\mathcal{A}_1$ (during election) and all computationally unbounded adversaries $\mathcal{A}_2^\infty$ (post-election) $\left| \Pr[\text{Exp}_{\text{VS}, \mathcal{A}}^{\text{priv}}(\lambda, 0) = 1] - \Pr[\text{Exp}_{\text{VS}, \mathcal{A}}^{\text{priv}}(\lambda, 1) = 1] \right| \leq \text{negl}(\lambda)$.

The public view $\text{view}_{\mathcal{A}} = (\text{pp}, \text{BB}, \text{result}, \{\delta_i\}_{i \in S})$ contains only the aggregate ciphertext C and its decryption, never individual vote ciphertexts. The (t, n) -threshold structure of VSS ensures that without t key shares, individual votes are information-theoretically hidden within the homomorphic sum, even against unbounded adversaries.

4.5 Dispute Resolution

Definition 13 (Dispute Resolution [16]). Let the mini-voting scheme $\Pi = (\text{Setup}, \text{Register}, \text{Vote}, \text{Tally}, \text{Verify})$ with authority Auth, voters $\mathcal{V}$, and bulletin board BB. Let $\text{Faulty}(\text{Auth}, b)$ denote a publicly verifiable set of traces where authority Auth is considered dishonest with respect to ballot bal . The individual dispute resolution properties are as follows.

- Cast ballot protection ($\text{VoterC}(\text{Auth})$):

$$\text{VoterC}(\text{Auth}) := \{tr \mid \text{verifyC}(H, \text{bal}) \in tr \Rightarrow (\exists [b]. \text{BB}_{\text{rec}}([b]) \in tr \wedge b \in [b]) \vee tr \in \text{Faulty}(\text{Auth}, b)\}$$
- Timeliness property ($\text{TimelyP}(\text{Auth})$):

$$\text{TimelyP}(\text{Auth}) := \{tr \mid \exists tr', tr''. tr = tr' \cdot tr'' \wedge \text{Ballot}(H, b) \in tr' \wedge \text{End} \in tr'' \Rightarrow (\exists [b]. \text{BB}_{\text{rec}}([b]) \in tr'' \wedge b \in [b]) \vee tr \in \text{Faulty}(\text{Auth}, b)\}$$

- *Abstaining voter protection* ($\text{VoterA}(\text{Auth})$):

$$\begin{aligned} \text{VoterA}(\text{Auth}) &:= \{tr \mid \text{verifyA}(H, b_H) \in tr \\ &\Rightarrow \neg(\exists [b], b. \text{BB}_{\text{rec}}([b]) \in tr \wedge b \in [b] \wedge \text{castBy}(b) \\ &= H \wedge b \notin b_H) \vee \exists b. tr \in \text{Faulty}(\text{Auth}, b)\} \end{aligned}$$
 - *Authority protection* ($\text{AuthP}(\text{Auth})$):

$$\text{AuthP}(\text{Auth}) := \{tr \mid \text{hon}(\text{Auth}) \in tr \Rightarrow \forall b. tr \notin \text{Faulty}(\text{Auth}, b)\}$$
- Protocol Π satisfies dispute resolution $\text{DR}(\Pi, T, p_H, p_{\text{Auth}}, p_f)$ in topology T if:
- $$\begin{aligned} \text{TR}(\Pi, T^{\text{Auth}+H+}) &\subseteq p_H \cap p_{\text{Auth}}, \quad \text{TR}(\Pi, T^{H+}) \subseteq p_H \\ \text{TR}(\Pi, T^{\text{Auth}+}) &\subseteq p_{\text{Auth}}, \quad \text{TR}(\Pi, T^{\text{Auth}+H+}) \cap p_f \neq \emptyset \end{aligned}$$
- where $T^{\text{Auth}+}$ denotes T with trusted authority, T^{H+} with trusted voter H , and p_f is a functional property.

Definition 14 (Faulty). For VoteSentinel, we define $\text{Faulty}(\text{Auth}, b)$ as the set of traces where:

1. Evidence exists that Auth received ballot b but b is not in recorded ballots:
 $\{tr \mid \exists \text{nullifier}_j, h_1, h_2, h_3, [b]. \text{Pub}(\text{Auth}, \text{BB}, (b, h_1, h_2, h_3, \text{nullifier}_j)) \in tr \wedge \text{BB}_{\text{rec}}([b]) \in tr \wedge b \notin [b]\}$
2. OR invalid nullifier resolution occurred:
 $\{tr \mid \exists \text{null}_v, j, j', b, b'. \text{nullifier}_j = H(\text{null}_v \| j) \in tr \wedge \text{nullifier}_{j'} = H(\text{null}_v \| j') \in tr \wedge j > j' \wedge b \in \mathcal{B}_{\text{final}}\}$

Theorem 5. Let T be the protocol topology with authority EA , trustees $\{T_i\}_{i \in [n]}$, and voters $\mathcal{V}$. The scheme VS satisfies dispute resolution $\text{DR}(\text{VS}, T, p_H, p_{\text{Auth}}, p_f)$ (Definition 13) where:

$$\begin{aligned} p_H &= \text{VoterC}(\text{EA}) \cap \text{TimelyP}(\text{EA}) \cap \text{VoterA}(\text{EA}) \\ p_{\text{Auth}} &= \text{AuthP}(\text{EA}) \cap \bigcap_{i \in [n]} \text{AuthP}(T_i) \end{aligned}$$

Under collision resistance of H in QROM, for evidence types $\text{type} \in \{\text{drop}, \text{bad-pdec}, \text{not-counted}, \text{bad-share}\}$: $\Pr[\text{Judge}(\text{pp}, \text{BB}, E) = (P, \text{err}) \wedge P \notin \text{Faulty}] \leq \text{negl}(\lambda)$.

The commitment $\gamma = H(c \| c' \| \nu_j \| \ell \| r)$ binds voters to their ballots, enabling them to prove ballot submission. The VSS commitments $\{W_{i,\ell}\}_\ell$ and key commitments $\Gamma_i = H(s_i \| \omega_i)$ enable detection of malicious trustees. The judge Judge algorithm verifies evidence against public commitments, assigning blame only when cryptographic proofs of misbehavior exist.

4.6 Accountability

Definition 15 (Accountability [17, 18]). Let the mini-voting scheme $\Pi = (\text{Setup}, \text{Reg}, \text{Vote}, \text{Tally}, \text{Vf}, \text{Judge})$ satisfy accountability if for all PPT adversaries $\mathcal{A}$ $\text{Adv}_{\mathcal{A}, \Pi}^{\text{acc}}(\lambda) = \Pr[\text{Exp}_{\Pi, \mathcal{A}}^{\text{acc}}(\lambda) = 1] \leq \text{negl}(\lambda)$, where the experiment $\text{Exp}_{\Pi, \mathcal{A}}^{\text{acc}}(\lambda)$ (Figure 5) returns 1 iff:

- **Fairness violation:** Judge blames honest party, i.e., $B \leftarrow \text{Judge}(\text{pd}, \text{BB}, E)$ with $B \neq \perp$ but $B \notin \text{Bad}(\text{pd}, \text{BB}, E, V)$, or
- **Completeness violation:** Result inconsistent with honest votes but no party blamed, i.e., $B = \perp$ but $|\text{BB}_{\text{vote}}| > N$ or $|\mathcal{F}| > N$ or honest votes not in result.

$\text{Exp}_{H,\mathcal{A}}^{\text{acc}}(\lambda)$	$\mathcal{O}_{\text{vote}}(j,v)$
1: $\text{pd} \leftarrow \mathcal{A}(1^\lambda); \quad V \leftarrow \emptyset$	1: $(\beta, \text{state}) \leftarrow \text{Vote}(\text{pp}, j, v)$
2: $(\text{BB}, \text{tL}) \leftarrow \mathcal{A}^{\mathcal{O}_{\text{vote}}}(\text{pd})$	2: $\sigma \leftarrow \text{ASCreate}(\text{ask}, \beta)$
3: for $(\beta, \sigma) \in \text{tL} : \text{ASVerify}(\text{pd}, \beta, \sigma) = 0$ then return 1	3: $V \leftarrow V \cup \{(\text{state}, \beta, \sigma)\}$
4: $E \leftarrow \{\text{VSDVerify}(s, \text{pd}, \text{BB}) : s \in V\} \cup \mathcal{A}(E)$	4: return (β, σ)
5: $B \leftarrow \text{Judge}(\text{pd}, \text{BB}, E); \quad B^* \leftarrow \text{Bad}(\text{pd}, \text{BB}, E, V)$	
6: return $(B \neq \perp \wedge B \notin B^*) \vee (B = \perp \wedge \neg \text{Valid}(V, \text{BB}))$	

Fig. 5: Accountability experiment where $\text{Valid}(V, \text{BB})$ checks result consistency.

Definition 16 (Evidence [18]). *Evidence E can be seen as a tuple set $(\text{type}, \text{data})$ which enables public verification of misbehavior, where the values of type can be assigned as $\text{type} \in \{\text{drop}, \text{not-counted}, \text{bad-pdec}, \text{bad-share}\}$.*

Theorem 6. *Let H be modeled by QROM. Assuming collision resistance of H and binding of Com against QPT adversaries, the voting scheme VS satisfies accountability (Definition 15). For all QPT adversaries $\mathcal{A}$: $\text{Adv}_{\mathcal{A}, \text{VS}}^{\text{acc}}(\lambda) = \Pr[\text{Exp}_{\text{VS}, \mathcal{A}}^{\text{acc}}(\lambda) = 1] \leq \text{negl}(\lambda)$ where $\text{Exp}_{\text{VS}, \mathcal{A}}^{\text{acc}}$ returns 1 iff (Judge blames honest party) $\vee$ (Judge returns $\perp$ on inconsistent result).*

Fairness holds because Judge only blames party P when verifiable evidence exists: commitment openings for voters, VSS verification failures for trustees, or partial decryption inconsistencies. Completeness holds because any deviation from honest execution ballot dropping, incorrect nullifier resolution, or malicious partial decryption produces publicly verifiable evidence against the responsible party.

5 Conclusion and Future Directions

We presented VoteSentinel, a post-quantum voting scheme achieving a wide range of properties. A notable design aspect is that VoteSentinel avoids heavyweight machinery: no zero-knowledge proofs are required for ballot validity, and no signature schemes are used for voter authentication. Instead, validity is enforced through constraints-the sum $\sum_{i=0}^{k-1} r_i = |\mathcal{F}|$ for single-choice or $\sum_{i=0}^{k-1} r_i = |\mathcal{F}| \cdot k(k-1)/2$ for ranked-choice voting-verified only after homomorphic aggregation. As future directions, we aim to eliminate the distributed key generation phase via silent-setup threshold cryptosystems [67]; however, current constructions rely on witness encryption, which remains impractical and quantum-vulnerable [68–70]. Designing a post-quantum silent-setup scheme supporting homomorphic operations is an open problem. As another future work, unlike other post-quantum schemes [3, 6, 8, 23, 53–57], we plan to deploy an actual production-ready implementation of VoteSentinel as an online web-based voting platform.

References

1. Daniel Rausch, Nicolas Huber, and Ralf Kusters. Verifiable E-Voting with a Trustless Bulletin Board . In *2025 IEEE CSF*, pages 457–472, Los Alamitos, CA, USA, June 2025. IEEE Computer Society.
2. Malte Bernhard and et al. Public evidence from secret ballots. In Robert Krimmer, Melanie Volkamer, Nadja Braun Binder, Norbert Kersting, Olivier Pereira, and Carsten Schürmann, editors, *Electronic Voting. E-Vote-ID 2017*, volume 10615 of *LNCS*. Springer, Cham, 2017.
3. Gilles Kaim, Sylvain Canard, Angèle Roux-Langlois, and Jacques Traoré. Post-quantum online voting scheme. In Malte Bernhard and et al., editors, *Financial Cryptography and Data Security. FC 2021 International Workshops. FC 2021*, volume 12676 of *LNCS*. Springer, 2021.
4. Ralf Kusters, Tomasz Truderung, and Andreas Vogt. Clash attacks on the verifiability of e-voting systems. In *2012 IEEE SP*, pages 395–409, 2012.
5. Josh Benaloh, Michael Naehrig, Olivier Pereira, and Dan S. Wallach. ElectionGuard: a cryptographic toolkit to enable verifiable elections. In *USENIX Security*, pages 5485–5502, Philadelphia, PA, August 2024. USENIX Association.
6. Rafieh MOSAHEB. *Quantum-Safe Electronic Voting Schemes*. PhD thesis, Université du Luxembourg, 2025.
7. Jillian Mascelli and Megan Rodden. “harvest now decrypt later”: Examining post-quantum cryptography and the data privacy risks for distributed ledger networks. Finance and Economics Discussion Series 2025-093, Board of Governors of the Federal Reserve System, Washington, D.C., 2025.
8. Patrick Hough, Caroline Sandsbråten, and Tjerand Silde. More efficient lattice-based electronic voting from NTRU. *IACR Communications in Cryptology*, 1(4), 2025.
9. Véronique Cortier, Joseph Lallemand, and Bogdan Warinschi. Fifty shades of ballot privacy: Privacy against a malicious board. In *2020 IEEE 33rd CSF*, pages 17–32, 2020.
10. David Bernhard, Oksana Kulyk, and Melanie Volkamer. Security proofs for participation privacy, receipt-freeness and ballot privacy for the helios voting scheme. In *Proceedings of the 12th International Conference on Availability, Reliability and Security*, ARES ’17. ACM, 2017.
11. Véronique Cortier, David Galindo, Ralf Küsters, Johannes Müller, and Tomasz Truderung. Sok: Verifiability notions for e-voting protocols. In *2016 IEEE SP*, pages 779–798, 2016.
12. Thomas Haines and Ben Smyth. Surveying definitions of coercion resistance. Cryptology ePrint Archive, Paper 2019/822, 2019.
13. Thomas Haines, Rafieh Mosaheb, Johannes Müller, and Ivan Pryvalov. Sok: Secure e-voting with everlasting privacy. *Proceedings on Privacy Enhancing Technologies*, 2023(1):279–293, 2023.
14. Maxime Meyer and Ben Smyth. Exploiting re-voting in the helios election system. *Information Processing Letters*, 143:23–28, 2018.
15. Feng Hao, Luke Harrison, Saverio Veltri, Irene Pugliatti, Chris Sinclair, and Gareth Nixon. Experience from UNITA elections: Reconciling revote, E2E verifiability and low coercion. Cryptology ePrint Archive, Paper 2025/1669, 2025.
16. David Basin, Saa Radomirovi, and Lara Schmid. Dispute resolution in voting. In *2020 IEEE CSF*, pages 1–16, 2020.
17. Mike et al. Graf. Accountable Bulletin Boards: Definition and Provably Secure Implementation . In *2024 IEEE CSF*, pages 201–216, Los Alamitos, CA, USA, July 2024. IEEE Computer Society.

18. Constantin-Cătălin et al. Drăgan. Machine-checked proofs of accountability: How to sEelect who is to blame. In Gene Tsudik, Mauro Conti, Kaitai Liang, and Georgios Smaragdakis, editors, *Computer Security – ESORICS 2023*, volume 14346 of *LNCS*. Springer, Cham, 2024.
19. David Bernhard, Véronique Cortier, David Galindo, Olivier Pereira, and Bogdan Warinschi. Sok: A comprehensive analysis of game-based ballot privacy definitions. In *2015 IEEE SP*, pages 499–516, 2015.
20. S. Delaune, S. Kremer, and M. Ryan. Coercion-resistance and receipt-freeness in electronic voting. In *19th IEEE CSFW’06*, pages 12 pp.–42, 2006.
21. IACR. IACR News Item: 2025 Election Update. <https://www.iacr.org/news/item/27138>, November 2025. Accessed: 2025-12-23.
22. J. Alex Halderman and Edward W. Felten. Secure, auditable, and usable voting with cryptography. In *USENIX Security*, SSYM ’02, pages 255–270. USENIX Association, 2002.
23. Xavier Boyen, Thomas Haines, and Johannes Müller. Epoque: Practical end-to-end verifiable post-quantum-secure e-voting. In *2021 IEEE EuroS&P*, pages 272–291, 2021.
24. Shweta Agrawal, Dan Boneh, and Xavier Boyen. Fast and compact subversion-resistant cryptosystems. In Henri Gilbert, editor, *Advances in Cryptology – EUROCRYPT 2010*, volume 6110 of *LNCS*, pages 257–276. Springer, 2010.
25. Carsten Baum, Ivan B. Damgård, Shengli Lu, K. Omkar, and Patrick Peer. Efficient lattice-based zero-knowledge proofs with improved concrete security. In Hovav Shacham and Alexandra Boldyreva, editors, *Advances in Cryptology – CRYPTO 2018*, volume 10991 of *LNCS*, pages 3–33. Springer, 2018.
26. Vadim Lyubashevsky. Lattice-based zero-knowledge proofs for integer relations. In Shai Halevi, editor, *Advances in Cryptology – EUROCRYPT 2012*, volume 7237 of *LNCS*, pages 92–110. Springer, 2012.
27. Damien Stehlé and Ron Steinfeld. Faster and more practical lattice-based cryptography. In Pil Joong Lee, editor, *Advances in Cryptology – EUROCRYPT 2011*, volume 6632 of *LNCS*, pages 237–261. Springer, 2011.
28. Diego F. Aranha, Dan Boneh, Rosario Gennaro, and Valerio Zappalà. Post-quantum lattice-based shuffle proofs and their applications. In Phong Q. Nguyen and Serge Vaudenay, editors, *Public-Key Cryptography – PKC 2021*, volume 12711 of *LNCS*, pages 297–326. Springer International Publishing, 2021.
29. Jonathan Bootle, Vadim Lyubashevsky, Cecilia Naya-Plasencia, and Tim Seiler. Exact zero-knowledge proofs for lattice-based cryptography. In Daniele Micciancio and Thomas Ristenpart, editors, *Advances in Cryptology – CRYPTO 2021*, volume 12826 of *LNCS*, pages 234–263. Springer International Publishing, 2021.
30. Sofiane Bouaziz, Maximilian Ermann, Eike Kiltz, Aline Labrabor, and Johannes Lehmkuhl. Efficient lattice-based blind signatures: Optimizations and implementation. IACR ePrint Archive, 2020. Cryptology ePrint Archive, Paper 2020/653.
31. Craig Gentry, Chris Peikert, and Vinod Vaikuntanathan. Trapdoors for hard lattices and new cryptographic constructions. In *STOC*, STOC ’08, pages 197–206. ACM, 2008.
32. Akshay Jain, Eike Kiltz, Chris Peikert, and Stefano Tessaro. New kinds of trapdoor functions from hard lattice problems. In Rei Safavi-Naini and Ran Canetti, editors, *Advances in Cryptology – CRYPTO 2012*, volume 7417 of *LNCS*, pages 692–710. Springer, 2012.
33. Aggarwal et al. Crystals-kyber: A quantum-resistant kem. *NIST Post-Quantum Cryptography Standardization*, 2023. Submission to the NIST Post-Quantum Cryptography Standardization Project.
34. Zvika Brakerski, Yuval Fan, and Frederik Vercauteren. Fully homomorphic encryption with a leveled structure. In *Proceedings of the 5th International Conference on Pairing-Based Cryptography*, volume 7281 of *LNCS*, pages 149–166. Springer, 2012.

35. Karim Bagheri. *II: A unified framework for computational verifiable secret sharing*. In Tibor Jager and Jia Pan, editors, *Public-Key Cryptography PKC 2025*, volume 15677 of *LNCS*. Springer, Cham, 2025.
36. Edouard Cuvelier, Olivier Pereira, and Thomas Peters. Election verifiability or ballot privacy: Do we need to choose? In Jason Crampton, Sushil Jajodia, and Keith Mayes, editors, *Computer Security ESORICS 2013*, volume 8134 of *LNCS*. Springer, 2013.
37. Josh Benaloh. Verifiable secret-ballot elections. 1987.
38. Ben Adida. Helios: web-based open-audit voting. In *USENIX Security*, SS’08, page 335348, USA, 2008. USENIX Association.
39. Véronique Cortier, Pierrick Gaudry, and Stéphane Glondou. Belenios: A simple private and verifiable electronic voting system. In Joshua Guttman, Carl Landwehr, José Meseguer, and Duško Pavlovic, editors, *Foundations of Security, Protocols, and Equational Reasoning*, volume 11565 of *LNCS*. Springer, Cham, 2019.
40. Sevdenur Baloglu, Sergiu Bursuc, Sjouke Mauw, and Jun Pang. Election verifiability revisited: Automated security proofs and attacks on helios and belenios. In *2021 IEEE CSF*, pages 1–15, 2021.
41. Veronique Cortier and Ben Smyth. Attacking and fixing helios: An analysis of ballot secrecy. In *2011 IEEE CSF*, pages 297–311, 2011.
42. Véronique Cortier and Pierrick Gaudry. Known caveats of belenios 3.1. Technical report, CNRS, Loria, April 2025. Version 3.1.
43. Ronald Cramer, Rosario Gennaro, and Berry Schoenmakers. A secure and optimally efficient multi-authority election scheme. In Walter Fumy, editor, *Advances in Cryptology — EUROCRYPT ’97*, pages 103–118, Berlin, Heidelberg, 1997. Springer.
44. ElectionGuard - Official — electionguard.vote. <https://www.electionguard.vote/spec/>. [Accessed 20-07-2024].
45. ElectionGuard - Official — electionguard.vote. https://github.com/microsoft/electionguard/releases/download/v2.1/EG_Spec_2_1.pdf. [Accessed 23-12-2024].
46. Pranav Baskar. Cryptographers held an election. they can’t decrypt the results. *The New York Times*, November 2025. Accessed: 2025-12-23.
47. BBC News. Cryptology firm cancels elections after losing encryption key. *BBC News*, November 2025. Accessed: 2025-12-23.
48. Eike Hauck, Eike Kiltz, Johannes Loss, and Ngoc Khanh Nguyen. Lattice-based blind signatures, revisited. In Daniele Micciancio and Thomas Ristenpart, editors, *Advances in Cryptology CRYPTO 2020*, volume 12171 of *LNCS*. Springer, Cham, 2020.
49. Andre Esser, Alexander May, and Floyd Zveydinger. McEliece needs a break – solving McEliece-1284 and quasi-cyclic-2918 with modern ISD. In *Advances in Cryptology – EUROCRYPT 2022*, volume 13277 of *Lecture Notes in Computer Science*, pages 433–457. Springer, 2022.
50. Ralf Küsters, Tomasz Truderung, and Andreas Vogt. Verifiability, privacy, and coercion-resistance: New insights from a case study. In *2011 IEEE S&P*, pages 538–553, 2011.
51. Ralf Küsters, Tomasz Truderung, and Andreas Vogt. Accountability: definition and relationship to verifiability. In *Proceedings of the 17th ACM Conference on Computer and Communications Security, CCS ’10*, page 526535. ACM, 2010.
52. Abapour et al. Blockchain-based economic voting with posterior security from lattices. In Carlos Cid and Naoto Yanai, editors, *IWSEC 2025*, volume 16208 of *LNCS*. Springer, Singapore, 2026.
53. Pavithra Sekar, J. Idhikash, Parth Agrawal, and J. R. Aarthy. Quantum-resistant electronic voting system using lattice cryptography techniques. In Maxim Bakaev, Radomir Bolgov, Anna Chizhik, Andrei Chugunov, Valeriia Demareva, Yury Kabanov, Roberto Pereira, R. Elakkiya, and Wei Zhang, editors, *Internet and Modern Society (IMS 2025)*, volume 2672 of *Communications in Computer and Information Science*, pages 257–270. Springer, Cham, 2026.

54. S. P. Mahanayak, B. Nikhita, and S. Bilgaiyan. Enhancing e-voting security with quantum-resistant encryption: A blockchain-based approach utilizing elliptic curve diffie-hellman and decentralized storage. *SN Computer Science*, 4(5):642, 2023.
55. Arome Junior Gabriel, Boniface Kayode Alese, Adebayo Olusola Adetunmbi, Olu-mide Sunday Adewale, and Oluwafemi Abimbola Sarumi. Post-quantum cryptography system for secure electronic voting. *Open Computer Science*, 9(1):292–298, 2019.
56. Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. A homomorphic LWE based e-voting scheme. In Tsuyoshi Takagi, editor, *Post-Quantum Cryptography – PQCrypto 2016*, volume 9606 of *Lecture Notes in Computer Science*, pages 245–265. Springer, Cham, 2016.
57. Rafaël del Pino, Vadim Lyubashevsky, Gregory Neven, and Gregor Seiler. Practical quantum-safe voting from lattices. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, CCS ’17*, page 15651581. Association for Computing Machinery, 2017.
58. Sunoo Park, Michael Specter, Neha Narula, and Ronald L. Rivest. Going from bad to worse: from internet voting to blockchain voting. *Journal of CyberSecurity*, 7(1):tyaa025, 2021.
59. Ralph C. Merkle. Secure communications over insecure channels. In Ralph C. Merkle, editor, *Proceedings of the Workshop on Computer Science*, pages 235–248. Stanford University, 1979.
60. Gilles Brassard, David Chaum, and Claude Crépeau. Minimum disclosure proofs of knowledge. In Gilles Brassard, editor, *Advances in Cryptology – EUROCRYPT ’87*, volume 304 of *LNCS*, pages 149–165. Springer, 1988.
61. Vadim Lyubashevsky, Chris Peikert, and Oded Regev. On ideal lattices and learning with errors over rings. In Henri Gilbert, editor, *Advances in Cryptology – EUROCRYPT 2010*, volume 6110 of *LNCS*, pages 1–23. Springer, 2010.
62. Valerio Cini, Giulio Malavolta, Ngoc Khanh Nguyen, and Hoeteck Wee. Polynomial commitments from lattices: Post-quantum security, fast verification and transparent setup. In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology – CRYPTO 2024*, volume 14929 of *LNCS*. Springer, Cham, 2024.
63. Xiayu Xiang, Shaoming Duan, Hezhong Pan, Peiyi Han, Jiahao Cao, and Chuanyi Liu. From one-hot encoding to privacy-preserving synthetic electronic health records embedding. In *Proceedings of the 2020 International Conference on Cyberspace Innovation of Advanced Technologies, CIAT 2020*, page 407413. ACM, 2021.
64. Julian R. Ullmann. Bit-vector algorithms for binary constraint satisfaction and subgraph isomorphism. *ACM J. Exp. Algorithmics*, 15, February 2011.
65. Ronald L. Rivest and Michael L. Dertouzos. On data banks and privacy homomorphisms. 1978.
66. Véronique Cortier and Joseph Lallemand. Voting: You can’t have privacy without individual verifiability. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, CCS ’18*, page 5366. ACM, 2018.
67. Sanjam Garg, Dimitris Kolonelos, Guru-Vamsi Policharla, and Mingyuan Wang. Threshold encryption with silent setup. In *Advances in Cryptology CRYPTO 2024*, page 352386. Springer, 2024.
68. Brent Waters and David J. Wu. Silent threshold cryptography from pairings: Expressive policies in the plain model. Cryptology ePrint Archive, Paper 2025/1547, 2025.
69. Mathias Hall-Andersen, Mark Simkin, and Benedikt Wagner. Silent threshold encryption with one-shot adaptive security. Cryptology ePrint Archive, Paper 2025/1384, 2025.
70. Jan Bormet and et al. BEAST-MEV: Batched threshold encryption with silent setup for MEV prevention. Cryptology ePrint Archive, Paper 2025/1419, 2025.